
Construcción y análisis de una función hash artesanal

Universidad Anáhuac - Blockchain

Santiago Cosme Cervera

29 de agosto de 2026

Resumen

Una función hash utilizada para aplicaciones reales garantiza su seguridad al cumplir con cuatro propiedades de seguridad: efecto avalancha, resistencia a colisiones, resistencia a preimagen y resistencia a segunda preimagen. Esta tarea propone una función hash artesanal y busca comprender de manera cuantitativa las propiedades de seguridad fundamentales a través de pruebas. No busco construir una función hash segura, si no que busco medir con rigor por qué no lo es.

1. Diseño

1.1. Definición formal de la función hash

La función hash propuesta se define de la siguiente manera

$$H : \{0, 1\}^* \rightarrow \{0, 1\}^{32} \quad (1)$$

donde cada mensaje m se convierte en una secuencia binaria y se procesa en bloques de 32 bits. La salida tiene una longitud fija de 32 bits, lo cual equivale a 8 dígitos hexadecimales.

Para convertir el mensaje original m a binario, primero obtenemos la codificación ASCII de cada carácter (cada símbolo se convierte en 8 bits). Así, si $m = c_1 c_2 \dots c_k$, entonces:

$$B(m) = \text{bin}(\text{ASCII}(c_1)) || \text{bin}(\text{ASCII}(c_2)) || \dots || \text{bin}(\text{ASCII}(c_k)) \quad (2)$$

1.2. Esquema de padding

Para asegurarse de que el mensaje quede dividido en bloques de 32 bits, se debe de aplicar un esquema de *padding*. Lo defino de la siguiente manera:

$$P(B) = B || 1 || 0^k || \text{len}(B)_{32} \quad (3)$$

donde 1 es un bit de marca, 0^k representa una secuencia de ceros y $\text{len}(B)_{32}$ la longitud original del mensaje. El valor de k se selecciona de manera que la longitud resultante sea múltiplo de 32.

1.3. Estado interno

El algoritmo mantiene un estado interno de 32 bits. Se inicializa con la constantes:

$$h_0 = 01101010000010011110011001100111 \quad (4)$$

equivalente al valor hexadecimal $0x6a09e667$. Este valor son los primeros 32 bits de la parte fraccionaria de $\sqrt{2}$ y sirve como el punto de partida para la actualización iterativa del digest.

Sea $P(B) = b_1 || b_2 || \dots || b_n$ con $|b_i| = 32$ para cada bloque b_i . El estado se actualiza en cada iteración mediante una función de compresión que toma como entrada el estado previo h_{i-1} y el bloque actual b_i .

1.4. Función de compresión

La función de compresión se define como una mezcla de tres operaciones (rotación circular hacia la izquierda que aporta asimetría, suma modular que aporta difusión y XOR que aporta no linealidad) sobre palabras de 32 bits. Para cada bloque b_i , el algoritmo lo convierte en un entero y ejecuta 8 rondas de mezcla (este parámetro puede cambiar). En cada ronda se aplica:

$$t_{j+1} = \text{ROTL}_3((h_{i-1} + t_j + j) \bmod 2^{32} \oplus 0x55555555) \quad (5)$$

donde:

- t_j es el valor intermedio en la ronda j
- j es el índice de la ronda
- $\oplus$ representa la operación XOR
- $ROTL_3$ denota una rotación circular hacia la izquierda de 3 posiciones
- $0x55555555$ es una constante fija de 32 bits

Después de completar las rondas, el nuevo estado se actualiza mediante

$$h_i = (h_{i-1} + t_8) \bmod 2^{32} \quad (6)$$

Finalmente, la salida del hash se obtiene como:

$$H(m) = \text{hex}(h_n) \quad (7)$$

2. Método

Para evaluar experimentalmente la calidad de la función hash, se analizaron cuatro propiedades fundamentales: efecto avalancha, resistencia a colisiones, resistencia a preimágenes y resistencia a segunda preimagen. En todos los casos se compara el valor experimental con su valor teórico esperado.

2.1. Efecto avalancha

Experimentalmente, se tomó un mensaje original y se alteró un único bit de su representación binaria. Luego se calculó el hash del mensaje original $H(m)$ y del nuevo mensaje alterado $H(m')$ y se midió la distancia de Hamming definida de la siguiente manera:

$$d_H = \sum_{i=1}^{32} [H_i(m) \neq H_i(m')] \quad (8)$$

El cálculo se repite para todos los bits del mensaje, y se calculó el promedio de bits alterados en la salida. En una función ideal, $E[d_H] = 16$ bits sobre 32, o un 50 % de los bits cambiados.

2.2. Colisiones

Experimentalmente, se truncó el digest a longitudes de salida de $d \in \{3, 4, 5, 6, 7\}$ dígitos hexadecimales. Para cada valor d , se generaron mensajes aleatorios y se calculó su digest truncado hasta encontrar dos mensajes distintos con el mismo valor. El número de intentos necesario para encontrar una colisión se registró y promedió sobre múltiples ejecuciones. El valor teórico esperado se aproxima mediante la "ley de cumpleaños":

$$E[\text{intentos}] \approx \sqrt{\frac{\pi}{2}} \cdot 2^{2d} \quad (9)$$

ya que el espacio de salida tiene tamaño 2^{4d} bits.

2.3. Resistencia a preimágen

Experimentalmente, se fijó un objetivo como un digest truncado $d \in \{3, 4, 5, 6, 7\}$ de longitud d , generado a partir de un mensaje previamente definido. Después se generaron mensajes aleatorios y se calculó su digest truncado hasta encontrar uno que coincidiera con el mensaje objetivo. La complejidad teórica esperada es

$$E[\text{intentos}] = 2^{4d} \quad (10)$$

dado el espacio de salida. Se requiere explorar aproximadamente la mitad del espacio antes de encontrar una preimagen.

2.4. Resistencia a segunda preimágen

Experimentalmente, se eligió un mensaje base m y se calculó su digest truncado. Luego, se generaron mensajes distintos de m y se buscaron coincidencias con el mismo digest objetivo. La expectativa teórica es la misma que para la preimagen:

$$E[\text{intentos}] = 2^{4d} \quad (11)$$

3. Resultados

A continuación se muestran los resultados de las pruebas de seguridad.

3.1. Prueba 1: Efecto avalancha

Mensaje	Long.	Intentos	$\bar{\delta}$ (bits)	% cam- bio	σ	[mín, máx]
<i>a</i>	8 b	8	15.38	48.05	1.77	[13, 18]
1	8 b	8	15.00	46.88	3.25	[10, 19]
<i>ok</i>	16 b	16	16.75	52.34	2.89	[13, 23]
<i>no</i>	16 b	16	17.56	54.88	2.73	[14, 22]
<i>sol</i>	24 b	24	16.33	51.04	2.99	[9, 22]
<i>hola mundo</i>	80 b	80	16.38	51.17	3.12	[9, 22]
<i>criptografia</i>	96 b	96	15.94	49.80	3.02	[9, 24]
<i>universidad anahuac</i>	152 b	152	15.99	49.98	3.13	[7, 24]
<i>laboratorio blockchain</i>	176 b	176	16.23	50.71	2.58	[7, 21]
<i>tarea artesanal</i>	120 b	120	16.00	50.00	2.57	[10, 22]
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
Global	—	4416	15.97	49.90	2.84	[7, 25]

Cuadro 1: *Efecto avalancha observado para distintos mensajes. La métrica $\bar{\delta}$ representa la media de bits cambiados en la salida al alterar un solo bit de la entrada.*

La función hash cumple con una buena difusión al obtener un valor global de 49.9 % de bits modificados, muy cercano del valor ideal de 50 % que deben de tener las funciones hash.

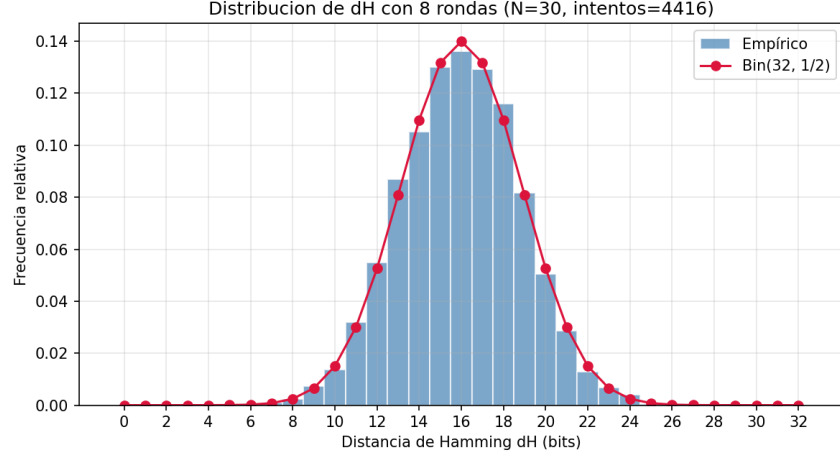


Figura 1: *Efecto avalancha observado al variar el número de rondas del algoritmo.*

Podemos observar en la *Figura 1* cómo la distribución experimental cumple $X \sim \text{Binomial}(32, 0.5)$

3.2. Prueba 2: Colisiones

d (hex)	n (bits)	Intentos teóricos	Intentos obs.	Tiempo medio (ms)	Repeti- ciones	Error rel. (%)
3	12	80.21	80.72	7.08	100	0.63
4	16	320.85	321.76	24.65	100	0.28
5	20	1283.39	1340.14	98.66	50	4.42
6	24	5133.57	6027.60	516.69	20	17.42
7	28	20534.30	19180.60	1600.80	10	6.59

Cuadro 2: *Resultados experimentales de la búsqueda de colisiones para distintos tamaños de digest truncado.*

Podemos observar que los resultados experimentales coinciden con los teóricos. El número de repeticiones fue bajo para $d \in \{5, 6, 7\}$ dado el cómputo que tenía disponible.

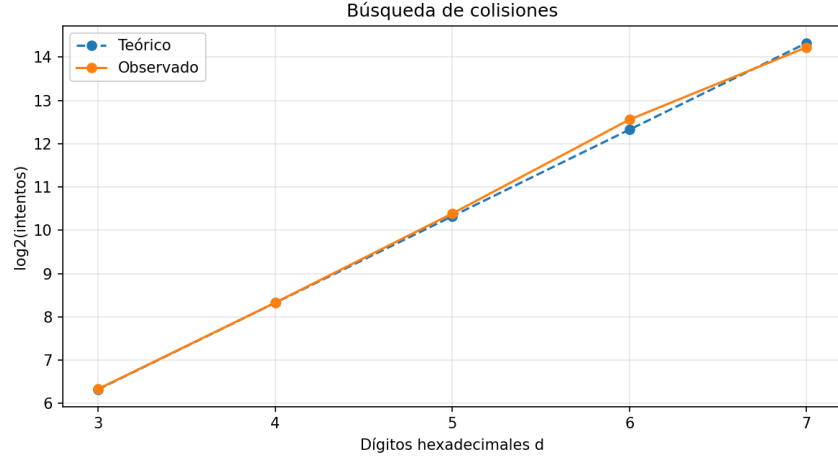


Figura 2: Comparación entre el costo teórico y observado para la búsqueda de colisiones.

Graficamente vemos la similaridad entre los valores teóricos y los observados.

3.3. Prueba 3: Resistencia a preimágen

3.4. Resistencia a preimágenes

d (hex)	n (bits)	Intentos teóricos	Intentos obs.	Mediana	Repeticiones
3	12	4096	4211.57	3023.0	100
4	16	65536	52958.11	42902.0	100
5	20	1048576	877756.47	563780.5	30
6	24	16777216	13479409.30	11948610.5	10
7	28	268435456	429821378.33	218920444.0	3

Cuadro 3: Resultados experimentales para la búsqueda de preimágenes a distintas longitudes de digest.

En este caso, podemos observar que los valores teóricos se alejan para $d \in \{6, 7\}$ dado que al tener cómputo limitado no fue posible correr el experimento tantas veces como era requerido para acercarse al valor teórico.

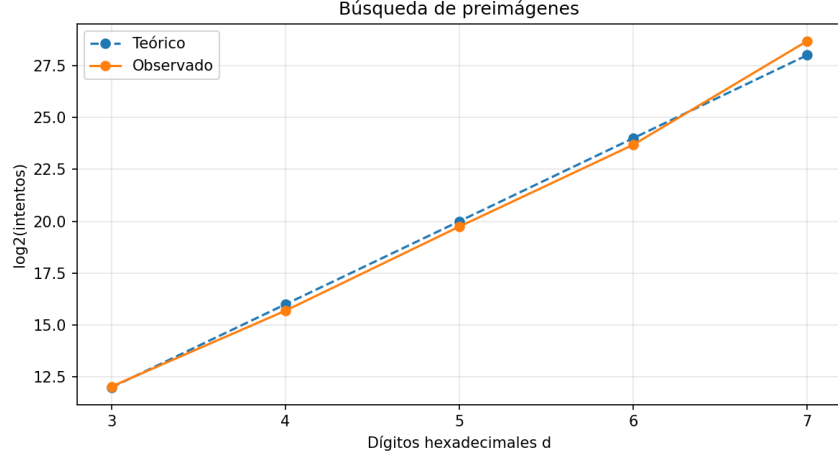


Figura 3: Comparación entre el costo teórico y observado para la búsqueda de preimágenes.

Para verificar si nuestra función hash es correcta en su construcción, realizamos la siguiente comparación.

$$\frac{T_{preimagen}(d)}{T_{colision}(d)} \quad (12)$$

Con los datos reales obtenidos para $d = 4$ tenemos:

$$\frac{52,958.11}{321.76} \approx 164.6 \quad (13)$$

El valor teórico está cerca de 200. No está muy desviado nuestro valor experimental pero para confirmar si la tendencia general es correcta, calculamos para $d = 5$.

$$\frac{877,756.47}{1340.14} \approx 655 \quad (14)$$

La tendencia es correcta: la preimagen se vuelve mucho más costosa que la colisión conforme aumenta d .

3.5. Prueba 4: Resistencia a segunda preimágen

d (hex)	n (bits)	Intentos teóricos	Intentos obs.	Tiempo medio	Repeticiones	$\rho(d)$
3	12	4096	4551.21	314.10 ms	100	1.081
4	16	65536	60255.43	4102.32 ms	100	1.138
5	20	1048576	950484.13	85071.10 ms	30	1.083
6	24	16777216	14245918.60	635488.60 ms	10	1.056
7	28	268435456	211628060.67	7121828.17 ms	3	0.492

Cuadro 4: *Comparación experimental entre la búsqueda de segunda preimagen y la de preimagen.*

donde $\rho(d)$ es igual a:

$$\rho(d) = \frac{T_{2^a \text{preimagen}}(d)}{T_{\text{preimagen}}(d)} \quad (15)$$

Los resultados experimentales coinciden con la teoría.

La teoría dice que ambos ataques deberían tener el mismo costo esperado, por lo que $\rho(d) \approx 1$. En los casos $d = 3, 4, 5, 6$ eso ocurre bastante bien: la razón está muy cerca de 1.

Intentamos encontrar defectos en el diseño utilizando las siguientes pruebas que no son de fuerza bruta.

Prueba	Qué se explota	Resultado
Relleno ambiguo	$H(M) \stackrel{?}{=} H(M\ 0x00\dots)$ si el padding no codifica la longitud original.	NO
Permutación de bloques	$H(m_1\ m_2) \stackrel{?}{=} H(m_2\ m_1)$. Si falla la conmutatividad, se rompe la estructura.	NO
Extensión de longitud	Dado $H(M)$, calcular $H(M\ X)$ sin conocer M . Debilidad clásica de Merkle-Damgård.	NO
Linealidad sobre $\mathbb{F}_2$	Buscar Δ tal que $H(M \oplus \Delta) = H(M)$ para todo M .	NO
Puntos fijos	Hallar (h, m) con $f(h, m) = h$ para insertar bloques sin alterar el digest.	NO

Cuadro 5: *Pruebas estructurales de segunda preimagen aplicadas al hash artesanal.*

Los resultados indican que no se encontraron fallas estructurales evidentes. Esto no significa que el hash sea seguro, sino que no parece presentar una debilidad estructural inmediata

4. Discusión y conclusiones

La comparación entre colisión, preimagen y segunda preimagen muestra una relación coherente con la teoría. La colisión resulta ser el ataque más accesible, mientras que la preimagen y la segunda preimagen requieren un costo mucho mayor. La Figura 4 resume esta tendencia. En ella se observa que la curva de colisión crece de manera más lenta que las curvas de preimagen y segunda preimagen, una diferencia que coincide con la expectativa teórica del modelo aleatorio. En otras palabras, el comportamiento empírico mantiene la jerarquía esperada de complejidades: la colisión es la propiedad más débil y la preimagen es la más costosa de atacar.

El cuadro 6 6 refleja este comportamiento de forma cuantitativa. En todos los casos evaluados, el costo observado crece exponencialmente con la longitud del digest, y la separación entre colisión y preimagen se mantiene de la misma manera que la teoría predice. Sin embargo, la diferencia entre los valores teóricos y observados en algunos puntos indica que la experimentación está influida por la variabilidad estadística y por el número limitado

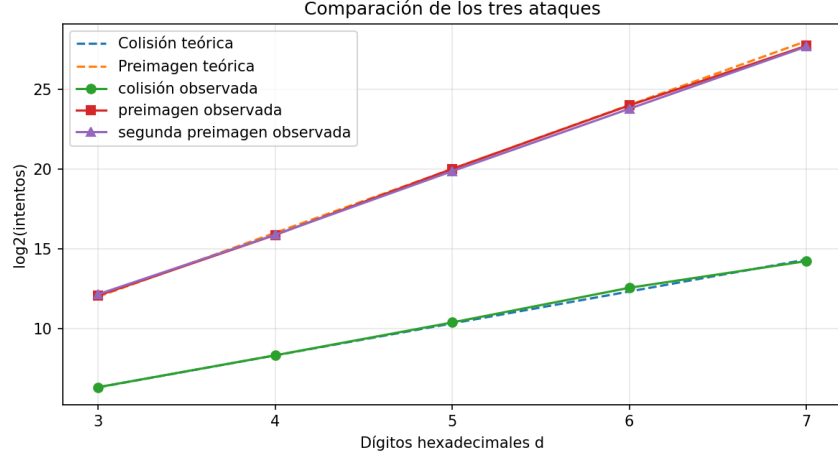


Figura 4: Comparación empírica de los costos observados para colisión, preimagen y segunda preimagen.

de repeticiones, particularmente para $d = 7$.

Ataque	El atacante elige	Costo ideal	Pendiente en d	Intentos ($d = 6$)
Colisión	M y M'	2^{2d}	2	$\approx 5,13 \times 10^3$
Segunda preimagen	solo M'	2^n	4	$\approx 1,68 \times 10^7$
Preimagen	nada	2^n	4	$\approx 1,68 \times 10^7$

Cuadro 6: Comparación del costo esperado para colisión, preimagen y segunda preimagen.

La evaluación de segunda preimagen estructural también es útil para establecer un marco de referencia: ninguna de las pruebas explotadas por ataque clásico fue efectiva. En particular, las pruebas de padding ambiguo, permutación de bloques, extensión de longitud, linealidad sobre $\mathbb{F}_2$ y puntos fijos no mostraron un camino real para construir una segunda preimagen sin costo significativo. Este resultado indica que el diseño no presenta una debilidad estructural evidente bajo las estrategias analizadas, aunque tampoco sugiere que pueda considerarse un algoritmo seguro para uso práctico.

En conclusión, la función propuesta funciona para entender la dinámica de una función hash y para medir propiedades fundamentales de seguridad. Los resultados experimentales muestran un comportamiento compatible con

el modelo aleatorio en varias dimensiones, especialmente en el efecto avalancha y en la escala creciente del costo para colisiones y preimágenes. Sin embargo, la implementación no cumple con los estándares de seguridad de sistemas criptográficos reales.